RECRUITER-GRADE SYSTEMS PROJECT

Low-Latency Trading Systems Lab

Blueprint complet - C++ / Linux / networking / concurrency / trading infrastructure / performance engineering

OBJECTIF

Construire une preuve d'ingenierie end-to-end: performance, correctness, failure handling, mesure et explicabilite.

- Release cible: dimanche 20 septembre 2026.
- Le projet reste le centre de gravite: 60% du temps de travail.
- LeetCode / algorithmique reste en maintenance: 20%, pas abandonne.

September 2026 build plan

Vue executive

REACTION VISEE "On le veut" ne vient pas du nombre de fichiers. Elle vient de la preuve que tu sais construire, casser, mesurer, profiler, expliquer et fiabiliser un systeme complet.

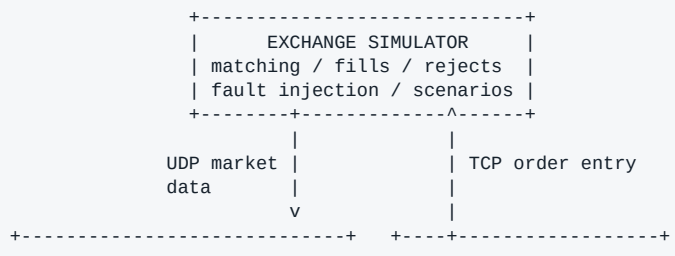
Axe	Ce que le repo doit prouver
C++ systems	RAII, memory layout, ownership, atomics, zero/minimal hot-path allocation, state machines.
Linux / networking	UDP multicast, TCP order entry, non-blocking I/O, epoll, socket options, CPU affinity, perf.
Trading infrastructure	Feed handler, sequencing, order book, risk, OMS, execution gateway, exchange simulator.
Reliability	Gap recovery, replay, snapshots, crash recovery, idempotency, fault injection, backpressure.
Performance engineering	Stage-level latency, p50/p99/p99.9, throughput, hardware counters, regression comparison.
Engineering maturity	Tests, reproducible scenarios, one-command demo, documentation, benchmark methodology.

Repartition de septembre

Travail	Part	Regle
Low-latency trading project	60%	Priorite absolue. Chaque semaine doit produire un chemin fonctionnel et mesurable.
Algorithmique / OA / LeetCode	20%	Maintenance ciblee: mediums chronometres, patterns deja appris, simulations OA.
C++ / Linux / perf cibles	10%	Apprendre seulement ce qui debloque ou explique le projet en cours.
Candidatures / LinkedIn / CV	10%	Visibilite progressive; ne pas attendre la release finale.

OUI, LEETCODE CONTINUE Mais il devient un filet de securite d'entretien. L'objectif n'est plus d'accumuler des patterns: c'est de rester rapide sur les screenings pendant que le projet construit ta differenciation.

Architecture end-to-end



CHEMIN DE DEMO Exchange -> UDP -> feed handler -> book -> strategy -> risk -> OMS -> TCP -> exchange -> ACK/FILL -> OMS, avec mesures et recovery observables.

1. Exchange Simulator

Le systeme doit parler a quelque chose de realiste et controllable.

- Programme separe: exchange_sim. Il publie le market data et recoit les ordres.
- Matching simplifie mais deterministe; generation de trades, ACK, rejects, partial fills, fills et cancels.
- UDP multicast pour le market data; TCP pour l'order entry.
- Scenarios reproductibles: marche calme, bursts, packet loss, disconnects, rejects, malformed messages.
- Seed explicite pour reproduire exactement un bug ou un benchmark.

Definition of done

- Le simulateur peut demarrer seul, accepter une connexion TCP, publier un flux UDP et rejouer un scenario deterministe.
- Un scenario provoque volontairement au moins: packet drop, duplicate, reorder, TCP disconnect et traffic burst.

2. Protocoles binaires

Mini protocole inspire des concepts ITCH/OUCH, sans copier un protocole proprietaire.

Market data	Order entry / execution
MessageHeader	NewOrder
AddOrder	CancelOrder
ModifyOrder	ReplaceOrder
CancelOrder	OrderAccepted
Trade	OrderRejected

Snapshot / Heartbeat	OrderFilled / OrderCancelled

- Binary layout explicite, framing, endianness, sequence numbers, validation, message type dispatch.
- Golden test vectors: bytes connus -> structure attendue -> re-serialization identique.
- Pas de JSON dans le hot path.

3. Networking Linux: UDP + TCP/IP

Le projet doit te faire manipuler le reseau pour de vrai.

UDP market data	TCP execution
multicast join / leave	connect / reconnect
SO_RCVBUF et reception non bloquante	partial reads / partial writes
epoll / receive loop	framing et heartbeats
sequence loss / reorder	timeouts et connection state
timestamps / batching experiment	TCP_NODELAY et socket buffers

- Comprendre le chemin Ethernet -> IP -> UDP/TCP -> socket -> kernel buffer -> userspace.
- Deux modes: real network mode entre processus; deterministic benchmark mode sans bruit reseau pour micro-benchmarks.
- Mesurer au lieu de generaliser: chaque optimisation est liee a un workload et une machine donnes.

4. Feed Handler + sequencing + recovery

Le premier vrai morceau d'infrastructure de marche.

```
packet -> validate header -> decode -> sequence check -> normalize -> publish event
```

- Detection de gap: expected=849, received=850 -> book stale -> recovery -> resume.
- Gestion des duplicates, out-of-order, stale packets et malformed messages.
- Snapshot + incremental replay ou replay cible des messages manquants.
- Compteurs explicites: gaps, duplicates, reordered, malformed, recovered.

5. Deterministic Replay Engine

Correctness, debugging et reproducibilite dans un seul mecanisme.

- Journaliser les inputs qui comptent: market events, orders, executions et state transitions.
- Une commande replay doit reconstruire le meme etat final a seed et inputs identiques.
- Comparer live/simulation vs replay: book, position, outstanding orders, sequence number.
- Le replay sert aussi de base aux tests de regression et a la reproduction d'incidents.

6. Order Book: baseline puis cache-aware

Ne pas annoncer qu'une structure est meilleure: le prouver sur le workload.

Version	But	Mesures
Baseline	Implementation simple et evidemment correcte, par ex. structure arborescente.	latency, throughput, allocations
Optimized	Representation plus contigue / preallouee / adaptee au range de prix.	p50/p99/p99.9, LLC misses, branch misses

EXPERIENCE FORTE Baseline vs cache-aware: meme input, meme machine, meme build; rapport automatique avec gains et regressions. La methodologie compte autant que le chiffre.

7. Concurrency Runtime

Single-thread ownership quand possible; communication explicite quand necessaire.

```
CORE 2: Network RX  -> SPSC ->  CORE 3: Book + Strategy  -> SPSC ->  CORE 4: OMS + Execution
```

- SPSC ring buffer lock-free borne; capacity fixe; comportement d'overflow defini.
- std::atomic, acquire/release, memory ordering documente par invariant, pas par magie.
- Cache-line alignment/padding; experience false sharing vs padded.
- CPU affinity; comparaison pinned vs unpinned.
- Backpressure: queue occupancy, drops/stalls, policy explicite sous surcharge.

8. Memory subsystem

La memoire est une partie de l'architecture, pas un detail.

- Preallocation au startup; fixed-capacity containers pour les chemins critiques lorsque pertinent.
- Object pool / arena / free list la ou le workload le justifie.
- Compteur d'allocations pour verifier le hot path au lieu de supposer qu'il est allocation-free.
- Mesurer page faults et cache behavior lorsque l'experience le demande.

```
Hot-path allocations
MarketData decode : 0
Book update       : 0
Strategy/Risk     : 0
Order creation    : 0   <- objectif a prouver, pas slogan
```

9. Strategy + Pre-trade Risk

La strategie est volontairement simple; l'infrastructure est la star.

- Signal deterministe minimal, par ex. book imbalance, uniquement pour faire traverser une decision end-to-end.
- Risk checks: max order size, max position, max notional, price sanity, duplicate order, kill switch.
- Mesurer le cout du risk: strategy->gateway risk OFF vs ON.
- Aucune revendication d'alpha ou de profitabilite necessaire.

10. OMS + Execution Gateway

Une vraie state machine, pas une liste de booleans.

```
CREATED -> PENDING_SEND -> SENT -> ACKNOWLEDGED
                                   |          |
                                   v          v
                              PARTIAL_FILL  CANCEL_PENDING
                                   |          |
                                   v          v
                                FILLED     CANCELLED

Terminal/exception states: REJECTED, TIMED_OUT
```

- clientOrderId et exchangeOrderId separes; qty, remainingQty, filledQty et etat coherents.
- Gestion des evenements impossibles: fill inconnu, duplicate ACK, reject apres fill, reconnect.
- Gateway non bloquante: framing, partial IO, heartbeat, timeout, reconnect.

- Round-trip mesurable: decision -> send -> ACK/FILL.

11. Persistence: Event Journal + Snapshot + Crash Recovery

Le systeme doit pouvoir mourir et revenir.

```
restart -> load snapshot -> replay journal tail -> reconstruct state -> reconcile -> READY
```

- Append-only event journal pour les transitions importantes.
- Snapshots periodiques avec version/schema explicites.
- Test visible: kill -9 pendant un scenario, restart, recovery, puis comparaison d'etat.
- Idempotency: le replay d'un evenement deja applique ne doit pas corrompre l'etat.

12. Fault Injection + Backpressure

Un systeme interessant est un systeme qu'on peut casser volontairement.

Faults a injecter	Observations attendues
packet drop / duplicate / reorder	gap detection, recovery, duplicate counter
TCP disconnect / delay	reconnect, timeouts, order reconciliation
malformed message	validation + reject/drop sans UB
traffic burst / queue saturation	occupancy, latency, drops/stalls, backpressure policy
engine restart	snapshot+journal rebuild, state equality

13. Performance Reporter

Le composant qui transforme le projet en preuve.

PRINCIPE Le Reporter ne doit pas perturber le hot path qu'il mesure. Instrumentation legere dans les threads critiques; aggregation/reporting hors du hot path.

Latence par stage

- network RX
- decode
- sequencing
- queue
- book update
- strategy
- risk
- OMS
- TCP send
- order->ACK
- end-to-end

Statistiques minimales

```
count | min | p50 | p90 | p99 | p99.9 | p99.99 | max
```

Throughput / queues / network / trading

- packets/s, messages/s, book updates/s, orders/s, fills/s.
- queue avg/max occupancy, overflow count, producer/consumer stalls.
- packets received/dropped, duplicates, out-of-order, sequence gaps, reconnects, bytes rx/tx.
- orders, ACKs, rejects, partial fills, fills, cancels, position, notional, risk rejects.

OS / hardware counters

- cycles, instructions, IPC, context switches, page faults, cache references/misses, branch instructions/misses.
- Importer ou capturer perf stat pour relier une modification de code a un effet materiel.

```
EXPERIMENT #41
Build      : Release / commit 72f51ab
Workload   : 10,000,000 messages / burst scenario
Pinning    : enabled

Stage            p50     p99     p99.9
Decode           ...     ...     ...
Book             ...     ...     ...
Risk             ...     ...     ...
End-to-end       ...     ...     ...

Correctness
Gaps recovered   ...
State mismatches 0
Lost orders      0
```

14. Experiment Runner + Performance Regression

Une optimisation doit etre reproductible et comparee.

```
./bench --scenario burst --book flat --queue spsc --pin-cpus --iterations 20
./compare baseline.json current.json
./report results/run_041
```

- Chaque run stocke config, commit, compiler/build, machine metadata, latency samples/histograms et counters.
- Regression report: delta p99/p99.9, throughput, allocations, cache/branch metrics.
- Flagger une regression meme si la moyenne s'ameliore.

Experiences a publier

Experience	Question
A. Order book	Tree-based baseline vs cache-aware representation?
B. Queue	mutex queue vs SPSC?
C. Affinity	unpinned vs CPU pinned?
D. Allocation	dynamic allocation vs preallocation?
E. Cache lines	false sharing vs padded?
F. Batching	batch 1 vs 8 vs 32?
G. Socket path	blocking vs non-blocking / epoll?

- Pour chaque experience: hypothesis -> methodology -> results -> interpretation -> limitations.

15. Testing, sanitizers et invariants

La correctness est un produit livrable.

```
Unit -> Protocol -> Book invariants -> OMS state machine -> Integration -> Replay -> Fault injection ->
Stress
```

- ASan et UBSan sur builds de validation; TSan sur scenarios adaptes.
- Invariants exemples: bestBid < bestAsk; filledQty <= originalQty; replayed state == pre-crash state.
- Fuzz/invalid-input tests simples sur le decoder si le temps le permet.

16. Control Plane + async logging

Separarer ce qui doit etre rapide de ce qui doit etre confortable.

Hot path	Control plane
market data -> book -> decision -> risk -> order	configuration
cheap timestamps/counters	report generation
bounded queues	snapshots
no std::cout / blocking disk IO	async logging / shutdown / diagnostics

- Async logger via queue preallouee; mesurer instrumentation ON/OFF.
- Observability ne doit pas devenir la plus grosse source de jitter.

17. Scenario Engine + one-command demo

Le recruteur doit comprendre le repo en quelques minutes.

- Config de scenario: duration, symbols, market rate, burst rate, packet faults, execution faults, seed.
- Commande unique: lance exchange + engine + scenario + fault + recovery + reporter.
- La demo finale doit imprimer un resume correctness + performance et generer un report.

```
$ ./run_demo.sh
[OK] 5,000,000 market events processed
[OK] gap detected and recovered
[OK] TCP reconnect completed
[OK] crash-recovery state matches
[OK] 0 state inconsistencies
Report: reports/demo/report.html
```

18. Repository cible

Une structure lisible vaut mieux qu'un mega dossier src/.

```
low-latency-trading-system/
├── engine/          market_data/ order_book/ strategy/ risk/ oms/ execution/
├── exchange/        matching/ market_data/ order_gateway/
├── network/         udp/ tcp/ epoll/
├── protocols/
├── concurrency/     spsc/ atomics/ memory/
├── persistence/     journal/ snapshot/ replay/
├── observability/   metrics/ histogram/ profiler/ reporter/
├── fault_injection/
├── benchmarks/
├── experiments/
├── scenarios/
├── tests/
```

```
├── tools/
└── docs/
```

19. Scope: MUST / SHOULD / v0.2

Le moyen de finir un gros projet sans fabriquer 17 demi-projets.

MUST SHIP avant le 20/09	SHOULD si avance	v0.2 apres release
Exchange simulator + UDP/TCP + binary protocol	Order book optimized plus poussee	Primary/standby replication
Feed handler + gap recovery	Allocation-free hot path plus strict	Multi-venue / multi-symbol advanced
Order book + SPSC + atomics + pinning	Async logger complet	NUMA / huge pages
Strategy + risk + OMS + gateway	Backpressure policies avancees	SO_BUSY_POLL / receive batching pousse
Journal + snapshot + crash recovery	Automated HTML report	Kernel-bypass / DPDK experiments
Fault injection + benchmark harness + Reporter	PCAP-style replay	Hardware timestamping / FPGA research
Tests + perf + README + one-command demo	Regression gate en CI	HA/failover approfondi

20. Definition of Done - release v1.0

Le 20 septembre, le systeme est livre si ces preuves existent.

Gate	Condition de sortie
End-to-end	Un scenario complet produit market data, decision, risk, order, ACK/FILL et etat OMS coherent.
Failure	Au moins packet gap, TCP disconnect, burst et crash/restart sont injectes et observes.
Recovery	Replay/snapshot reconstruit un etat identique selon des invariants automatises.
Performance	Reporter produit p50/p99/p99.9 + throughput + queue/network counters.
Profiling	Au moins 3 experiences documentees avec methodology + before/after.
Quality	Tests verts; sanitizers lances sur builds adaptes; aucune UB connue.
Usability	Build documente + one-command demo + README architecture/results.
Reproducibility	Scenario seed, config, commit et machine metadata stockes avec chaque run.

21. Packaging GitHub / LinkedIn / CV

La visibilite commence avant la fin.

Moment	Contenu public
Post 1 - debut	Architecture, ambition, protocol/network path, repo skeleton.
Post 2 - market data	Gap detection/recovery + feed handler + first benchmark.
Post 3 - concurrency/book	SPSC / cache locality / before-after benchmark.
Post 4 - resilience/reporting	Fault injection, recovery, p99/p99.9, Reporter.
Final - 21/22 sept. recommande	Full architecture, demo, GitHub, strongest results, lessons, next steps.

CV bullet cible

VERSION COURTE Built an end-to-end C++/Linux low-latency trading systems lab with UDP multicast market data, TCP order entry, binary protocol decoding, SPSC pipelines, cache-aware order-book processing, pre-trade risk, OMS, deterministic replay/crash recovery and an automated p50/p99/p99.9 performance reporter.

Ce que tu dois pouvoir expliquer en entretien

- Pourquoi UDP pour le feed et TCP pour l'order entry dans ton modele.
- Pourquoi le SPSC est correct, quel invariant justifie acquire/release.
- Pourquoi une structure de book a gagne/perdu sur TON workload.
- Ou se trouve le jitter; ce que p99.9 revele que la moyenne cache.
- Que se passe-t-il sous packet loss, disconnect, backlog et crash.
- Comment tu evites que logging/reporting fausse la mesure.
- Quelle optimisation tu as abandonnee parce que les chiffres ne la justifiaient pas.

La phrase qui resume le projet

NARRATIVE "I did not build a toy trading strategy. I built a small, measurable trading infrastructure: I can feed it, execute through it, overload it, disconnect it, crash it, recover it, replay it and show exactly where the latency went."

C'est cette combinaison - architecture + correctness + performance + preuves - qui doit fabriquer le "On le veut".